ARAŞTIRMA SORULARI 12 EKİM

1.Browser/client nedir? Server nedir?

-Browser(tarayıcı): İnternet sitelerini açmamızı sağlayan programdır.Örneğin: Chrome,Firefox ve Edge.

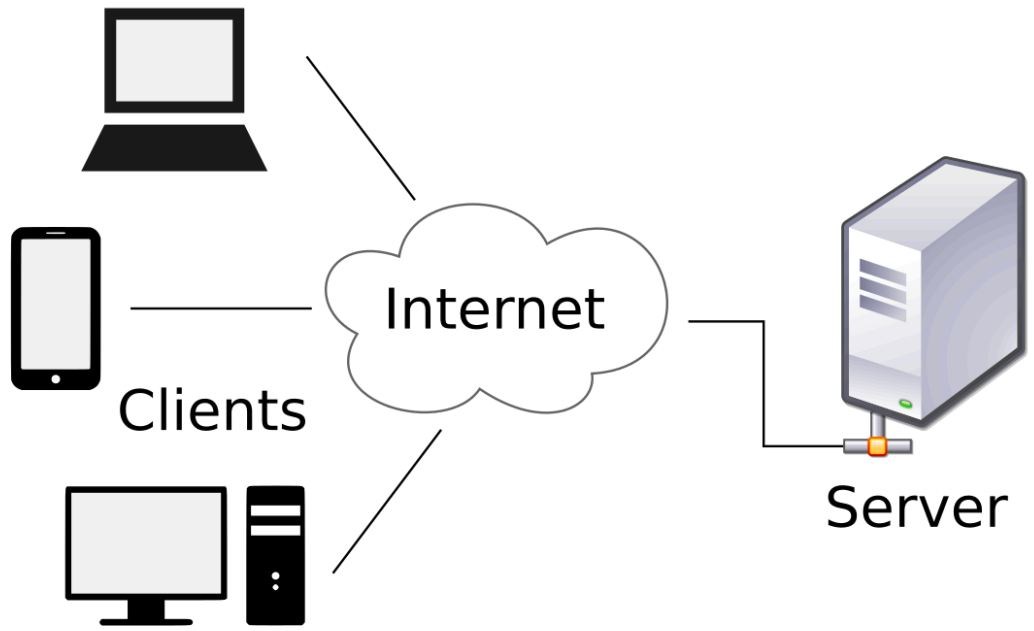
-client(istemci): Sunucudan bilgi veya hizmet isteyen cihaz ya da programdır.Bilgisayarımız veya telefonumuz buna örnektir.

-Servet(sunucu): İstemcilerden gelen istekleri karşılayan ve onlara veri veya hizmet sağlayan bilgisayar ya da yazılımdır.

-Nasıl çalışırlar?:

Client / İstemci

Bilgisayar veya telefon



İstek gönderilir, cevap alınır.

Server / Sunucu

İsteği karşılar ve cevap gönderir.

2.Frontend ve backend hangi sorumlulukları üstlenir?

Frontend(ön uç) Sorumlulukları:

-Kullanıcı Arayüzü (UI) ve Deneyimi (UX):
–Butonlar,menüler,formlari,sayfaların tasarımıbve görsel düzeni.

-İstemci Tarafı Etkileşim:
–Kullanıcının tıkladığı, yazdığı veya kaydırdığı anlarda gerçekleşen anlık hareketler.

-Responsive (uyumlu) tasarım:
–Uygulamanın telefon,tablet ve bilgisayar ekranlarında düzgün görünmesi.

-Performans ve erişilebilirlik:
–Sayfaların hızlı yüklenmesi ve her kullanıcının rahatça erişebilmesi.

-Kullanılan teknolojiler:
–HTML,CSS,JavaScript ve bunlara bağlı framework'ler.

backend(arka uç) sorumlulukları:

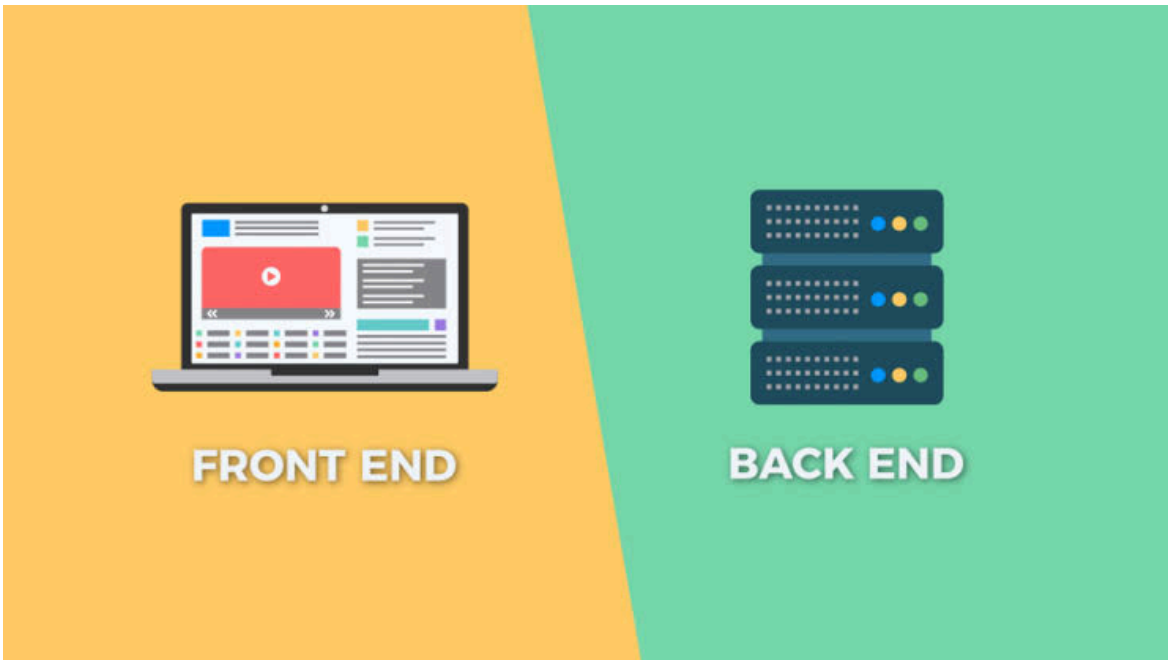
-Veritabanı yönetimi:
–Kullanıcı bilgileri,ürünler veya yazının saklanması, güncellenmesi ve çağrılması.

-API geliştirme:
–Fronted ile backend arasında veri alışverişini sağlayan köprüleri kurmak.

-İş mantığı(Business logic):
–Üyelik doğrulama, ödeme işleme, yetkilendirme ve kural tabanlı tüm hesaplamalar.

-Güvenlik
–Verilerin şifrelenmesi, yetkisiz erişimlerin engellenmesi ve sunucu koruması.

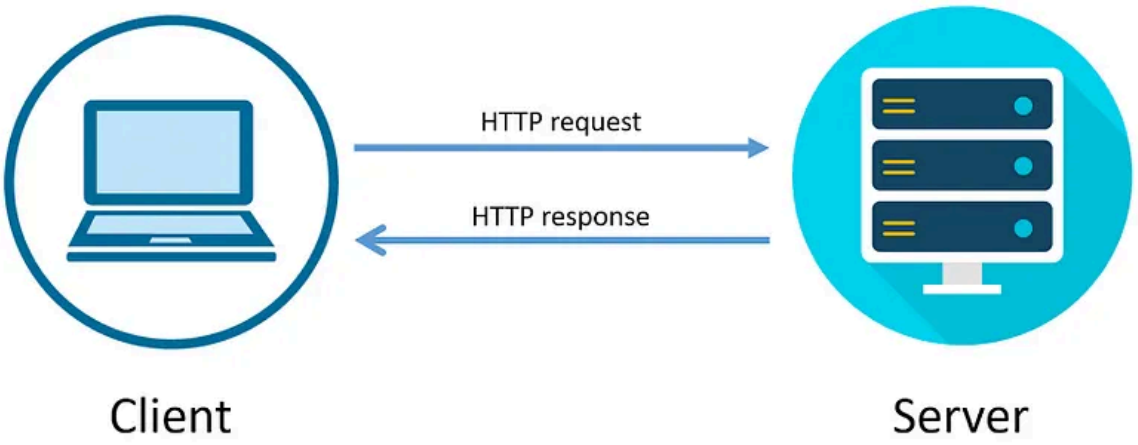
-Kullanılan teknolojiler:
–Node.js, python, java, PHP, C# (.NET) ve veritabanı sistemleri.



3.HTTP request ve response nedir?

HTTP request: Client(istemci) ve server(sunucu) arasındaki istek ve yanıt mesajları ile sağlanır.Client, server'a bir kaynağı erişmek için istekte bulunur.

HTTP response: Bir isteğe karşılık server tarafından client tarafına verilen yanıt.



4.GET ve POST hangi durumlarda karşımıza çıkar?

GET methodu kullanıldığında istekler URL kısmında gönderilir. Gönderilen bilgiler URL'de görüntülendiği için Güvenlik riski yüksektir ancak POST metoduna göre daha hızlıdır.

/test/demo_form.php?name1=value1&name2=value2

POST metodu ise URL'den görüntülemesi istemediğimiz istekler için kullanılır.Gönderilen bilgiler URL'de görünmediği için daha gübenlidir ancak GET methoduna göre daha yavaştır. Yani POST metoduna form alanları HTTP REQUEST HEADER içine kodlanmış olarak gider, URL'de gözükmez.POST işleminde her türlü dosyayı post edebiliriz.

GET'in POST'a karşı bir diğer dezavantajı şudur: multi-part binary türündeki verilerin gönderimi. "multi-part binary" ile kastedilen şey dosyadır. Örneğin; resimdir. POST ile forma bir resim koyup sunucuya upload edebiliriz. Fakat GET ile bu mümkün değildir.

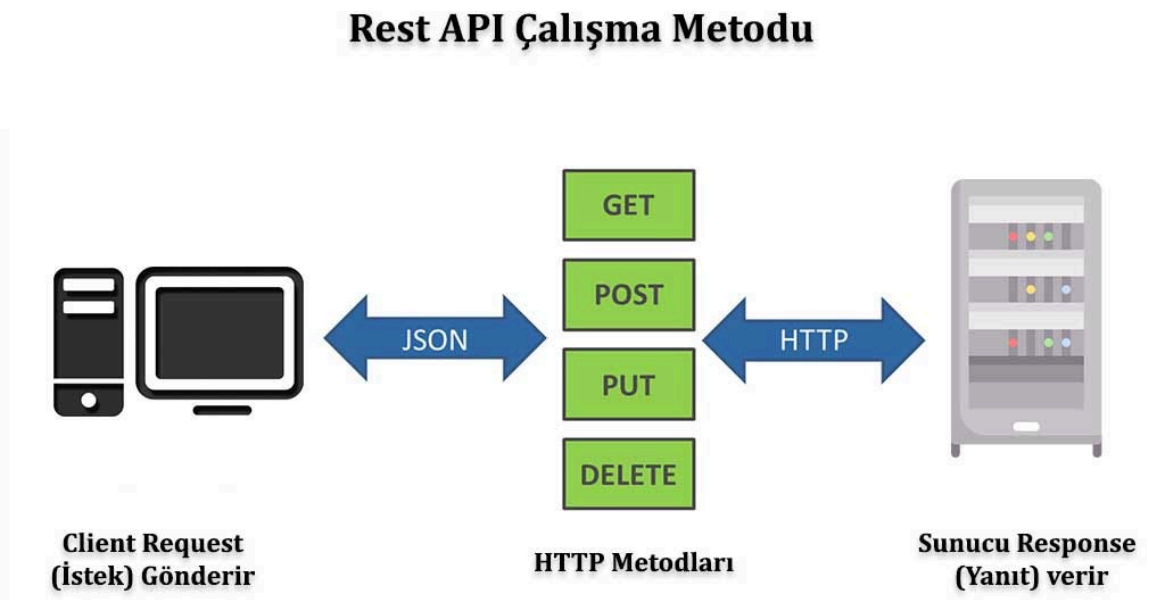
	GET	POST
Geri Butonu, Sayfa Yenileme Butonu	Varılan sayfa kaybolmaz.	Varılan sayfa için Re-Submit popup'ı görüntülenir.
Yer İmleme	Mümkün	Mümkün değil
Önbelleğe Kaydedilme	Mümkün	Mümkün değil
Kodlama Tipi(Encoding Type)	application/x-www-form-urlencoded[1]	application/x-www-form-urlencoded[1] ya da multipart/form-data[2]
Tarayıcı Geçmişi	Değişkenler tarayıcı geçmişinde kalır	Değişkenler tarayıcı geçmişine kaydedilmez
Karakter Limiti	2048 karakter	Limit yok
Veri Tipi Kısıtlaması	Sadece ASCII karakterleri izinlidir	Sınırlama yoktur(Binary tipli veri izinlidir)
Güvenlik	GET POST'a göre daha az güvenlidir, çünkü gönderilen veri URL'nin bir parçasını teşkil eder.	POST GET'e göre biraz daha güvenlidir, çünkü gönderilen değişken ve değerleri tarayıcı geçmişinde ya da sunucu log'larında depolanmaz.
Görünürlük	URL'deki veri herkes tarafından görünürdür.	Veri URL'de gösterilmez.

5.API nedir? JSON neden sık görülür?

API,Uygulama programlama arayüzü anlamına gelir,ancak son zamanlarda daha çok web API'si ile eş anlamlı hale gelmiştir.Genellikle API oluşturmak için REST kullanılır ve bu da API ile istemci arasındaki iletişim için standart HTTP protokolünü kullanılır.

JSON,JavaScript nesne gösterimi anlamına gelir ve nesnelerin JavaScript'te nasıl gösterildiğini belirtir.Dolayısıyla,bir API JSON veri formatıyla yanıt veriyorsa,HTTP

başlığındaki Content/type değerini application/json veya text/json olarak ayarlar.



6.DNS kabaca hangi problemi çözer?

DNS: Herhangi bir dijitalin mülkün giriş kapısı olarak bildiğimiz “www” ile başlayan adresler, aslında arka planda çok daha büyük ve rakamsal bir sistemin görünen yüzünü anlatır. Sayılardan meydana gelen bu IP adresleri,DNS sisemleri tarafından oluşturulmaktadır.

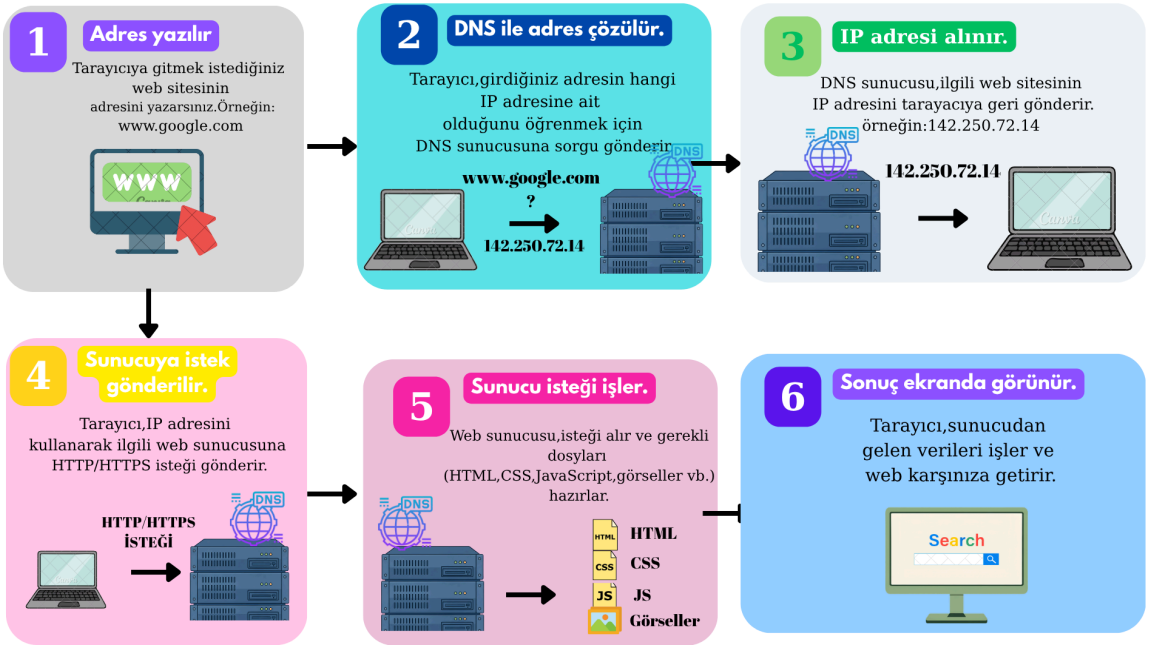
Hangi problemi çözer?
-DNS, bilgisayarların birbirini bulmak için kullandığı karmaşık sayısal adresleri (IP adreslerini), insanların kolayca hatırlayabilceği web sitesi isimlerine (alan adlarına)

dönüştürme problemini çözer.



Gün sonunda teslim edeceğiniz not

1.Tarayıcıya bir adres yazmanızdan ekranda sonuç görmenize kadar olan akışı çizin:



2.Fronted,backend,server,request,response,API ve JSON kavramlarını aynı senaryo içinde ilişkilendirin:

Bir online alışveriş satın alma sitesi senaryosu üzerinden bu kavramları birbirine bağlayalım.
-Senaryo:Ayakkabı satın alma sitesi

1.Frotend(ön yüz):Kullanıcının gördüğü site tasarımıdır.Ayakkabı fotoğrafları,fiyatlar ve satın alma butonu fronted kısmıdır.

2.Request(istek):Kullanıcı"ürünleri göster"butonuna bastığında tarayıcı,ürünleri getirmek için sunucaya bir istek gönderir.

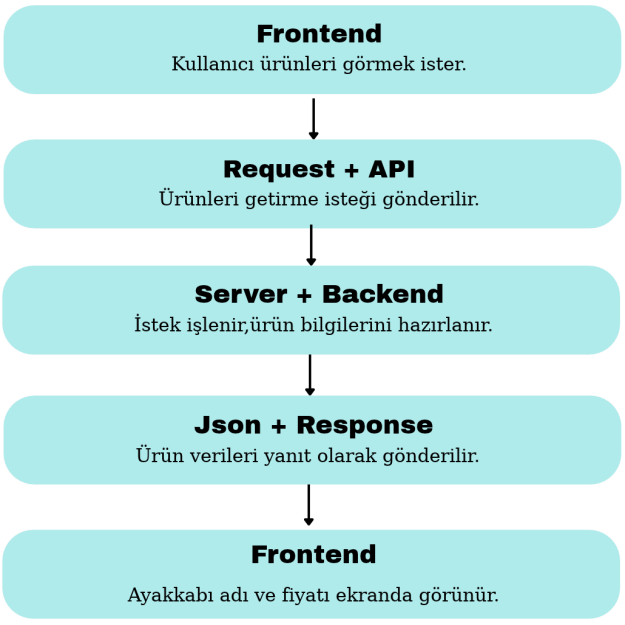
3.Server(sunucu):Gelen isteği karşılayan ve gerekli işlemlerin yapılmasını sağlayan sistemdir.

4.Backend(arka yüz):Ürün bilgileri veri tabanından alır ve gerekli işlemleri gerçekleştirir.

5.API:Fronted ile backend arasında iletişim kurulmasını sağlayan arayüzdür.

6.JSON:Ürün bilgilerinin aktarılmasında kullanılabilen veri formatıdıri

7.Response(yanıt):Sunucu,ürün bilgilerini içeren yanıtı tarayıcıya gönderir.



Kısaca:
Frontend kullanıcıya görünen kısımdır,backend işlemleri yapar,server istekleri karşılar,API iletişimi sağlar,request istektir,response yanıttır ve JISON verilerin düzenli aktarılmasını sağlar.

3.Bir web isteğinin nerede başlayıp nerede bittiğini kendi cümlenizle anlatın:

–Bir web isteği,kullanıcının tarayıcıda bir butona basması veya bir web adresine girmesiyle başlar.Tarayıcı,sunucuya bir request(istek)gönderir.Sunucu isteği işler ve response(yanıt)göderir.Tarayıcı bu yanıtı işleyip sonucu ekranda gösterdiğinde istek süreci tamamlanır.